eda.ipynb

Exploratory Data Analysis

Steel Plates Faults

```
from hashlib import sha256
from pathlib import Path
from urllib.request import Request, urlopen
from zipfile import ZipFile
import shutil

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")
pd.set_option("display.max_columns", 40)
```

Скачаем датасет (https://archive.ics.uci.edu/dataset/198/steel+plates+faults) из UCI.

```
data_url = "https://archive.ics.uci.edu/static/public/198/steel+plates+faults.zip"
expected_sha256 = "cb8eb9859198b63f053e443513036b401746fa517ef58bd17c846c6741c93919"
data_path = Path("data/steel_plates_faults.zip")
data_path.parent.mkdir(exist_ok=True)

if not data_path.exists():
    request = Request(data_url, headers={"User-Agent": "Mozilla/5.0"})
    with urlopen(request) as response, data_path.open("wb") as output:
        shutil.copyfileobj(response, output)

assert sha256(data_path.read_bytes()).hexdigest() == expected_sha256
```

```
with ZipFile(data_path) as archive:
    columns = archive.read("Faults27x7_var").decode().splitlines()
    data = pd.read_csv(archive.open("Faults.NNA"), sep=r"\s+", names=columns)

target_columns = [
    "Pastry", "Z_Scratch", "K_Scatch", "Stains",
    "Dirtiness", "Bumps", "Other_Faults"
]

data["fault_type"] = data[target_columns].idxmax(axis=1)
df = data.drop(columns=target_columns)
```

```
df.head()
```

```
df.shape
```

```
df.info()
```

```
df.isna().sum().sum(), df.duplicated().sum()
```

Target

```
df["fault_type"].value_counts()
```

```
order = df["fault_type"].value_counts().index

plt.figure(figsize=(10, 5))
sns.countplot(data=df, y="fault_type", order=order)
plt.xlabel("count")
plt.ylabel("fault type");
```

Классы представлены неравномерно. Посмотрим на их доли.

```
df["fault_type"].value_counts(normalize=True).round(3)
```

Numerical features

```
df.describe().T
```

Посмотрим на распределения нескольких признаков.

```
features = [
    "Pixels_Areas", "X_Perimeter", "Y_Perimeter",
    "Minimum_of_Luminosity", "Maximum_of_Luminosity",
    "Steel_Plate_Thickness", "Edges_Index", "Luminosity_Index"
]

df[features].hist(figsize=(16, 10), bins=30)
plt.tight_layout();
```

У `Pixels_Areas` длинный правый хвост. Сравним исходный и логарифмический масштабы.

```
fig, axes = plt.subplots(1, 2, figsize=(14, 4))

sns.histplot(df["Pixels_Areas"], bins=50, ax=axes[0])
axes[0].set_title("Pixels_Areas")

sns.histplot(np.log10(df["Pixels_Areas"]), bins=50, ax=axes[1])
axes[1].set_title("log10(Pixels_Areas)");
```

Exercise 1: feature detective

В датасете есть `LogOfAreas`, `Log_X_Index` и `Log_Y_Index`. Восстановите формулы, по которым они были получены, и проверьте догадку на всех строках.

```
# YOUR CODE
```

Outliers

```
fig, axes = plt.subplots(1, 3, figsize=(16, 4))

for feature, ax in zip(["Pixels_Areas", "X_Perimeter", "Y_Perimeter"], axes):
    sns.boxplot(data=df, x=feature, ax=ax)

plt.tight_layout();
```

Посчитаем, сколько значений лежит за границами $1.5 \cdot IQR$.

```
def count_iqr_outliers(series):
    q1, q3 = series.quantile([0.25, 0.75])
    iqr = q3 - q1
    return ((series < q1 - 1.5 * iqr) | (series > q3 + 1.5 * iqr)).sum()

df[["Pixels_Areas", "X_Perimeter", "Y_Perimeter"]].apply(count_iqr_outliers)
```

Теперь посмотрим на площадь отдельно для каждого типа дефекта.

```
plt.figure(figsize=(12, 6))
sns.boxplot(data=df, x="Pixels_Areas", y="fault_type", showfliers=False)
plt.xscale("log");
```

```
df.groupby("fault_type")["Pixels_Areas"].agg(["count", "mean", "median",
"max"]).sort_values("median")
```

Большая площадь характерна для `K_Scatch`. Такие значения нельзя удалять только потому, что они оказались за границей boxplot.

Exercise 2: delete the outliers?

Удалите строки, в которых `Pixels_Areas` лежит выше границы $Q_3 + 1.5 \cdot IQR$. Посчитайте, какую долю каждого типа дефекта мы потеряем.

```
# YOUR CODE
```

Correlation

```
corr_features = [
    "Pixels_Areas", "X_Perimeter", "Y_Perimeter",
    "Sum_of_Luminosity", "Minimum_of_Luminosity",
    "Maximum_of_Luminosity", "Steel_Plate_Thickness",
    "Edges_Index", "Square_Index", "Luminosity_Index"
]

fig, axes = plt.subplots(1, 2, figsize=(18, 7))
sns.heatmap(df[corr_features].corr(method="pearson"), vmin=-1, vmax=1, cmap="coolwarm",
ax=axes[0])
sns.heatmap(df[corr_features].corr(method="spearman"), vmin=-1, vmax=1, cmap="coolwarm",
ax=axes[1])
axes[0].set_title("Pearson")
axes[1].set_title("Spearman")
plt.tight_layout();
```

Посмотрим на связь площади и периметра.

```
plt.figure(figsize=(11, 7))
sns.scatterplot(
    data=df,
    x="X_Perimeter",
    y="Pixels_Areas",
    hue="fault_type",
    alpha=0.65
)
plt.xscale("log")
plt.yscale("log")
plt.legend(bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout();
```

Exercise 3: correlation can change

Посчитайте Spearman correlation между `Edges_Index` и `X_Minimum` на всех данных, а затем отдельно для каждого типа дефекта. Сравните результаты.

```
# YOUR CODE
```

```

```